

CV大模型系列之：多模态经典之作CLIP，探索图文结合的奥秘

猛猿 2023-08-07 4,938 阅读14分钟

关注

⚠⚠⚠ 本文为稀土掘金技术社区首发签约文章，30天内禁止转载，30天后未获授权禁止转载，侵权必究！

在本系列之前的文章中，我们曾经讲过VIT（Vision Transformer），一个借助Transformer Encoder架构来实现图片分类的模型。由于VIT成功证明了摆脱CNN，完全在语言模型架构上做CV任务的可能，因此它也开启了多模态模型研究的大门。

所谓多模态，就是指不同领域的输入数据，比如文字、图片、语音、视频等等。在传统方法中，每个领域都有一些经典的处理算法，比如用于处理文本的RNN，LSTM，Transformer，用于处理图像的各类卷积神经网络等，**各领域间相对独立**。但是，**人们总会遇上需要联合领域数据的时候**：比如给一张图片，输出一段关于这个图片的描述；或者给一段文字，输出一张符合文字描述的图片。**而实现这一目标的难点在于**：不同领域数据间的特征分布、特征信息是不一样的。**因此多模态模型的总体目标就是**：训练一个模型，一方面能统一特征表达，另一方面又能让不同模态特征间学到相关性。

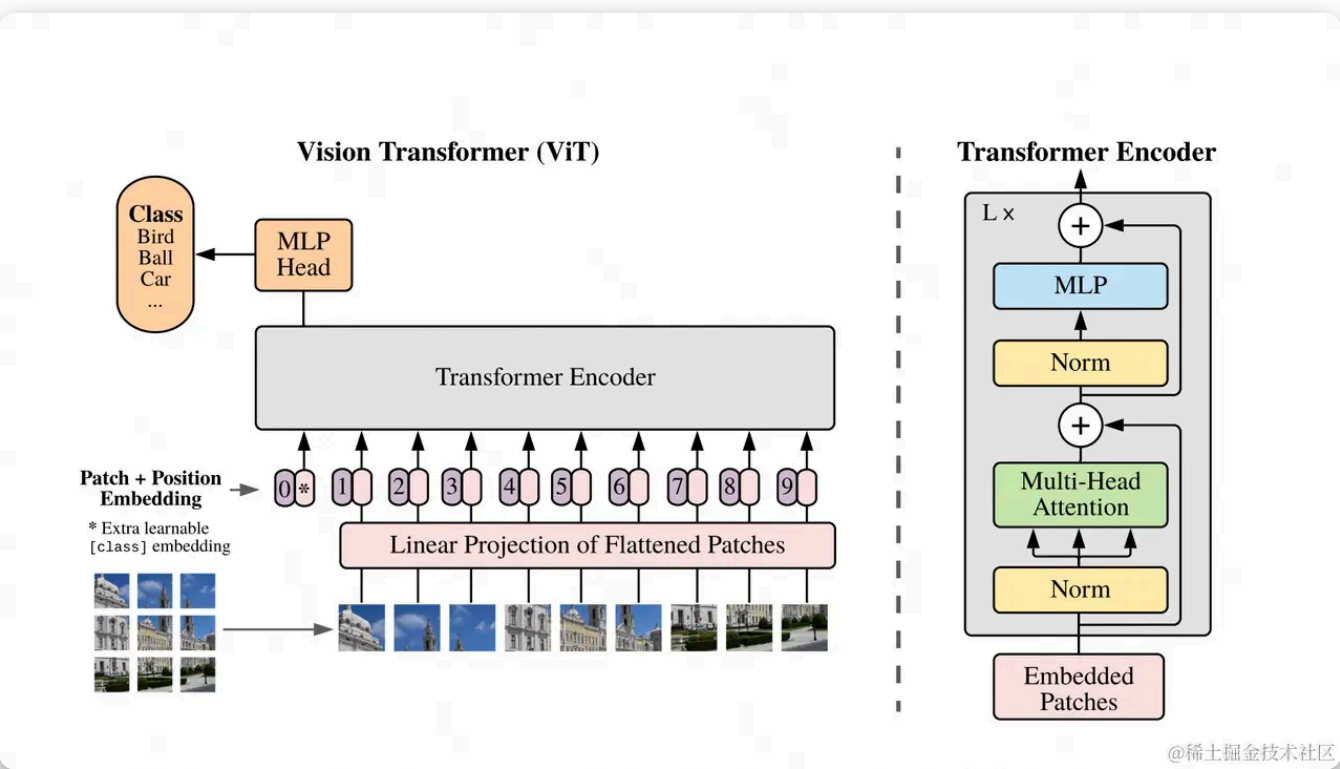
在这篇文章中，我们将来解读OpenAI提出的多模态模型：**CLIP（Contrastive Language-Image Pre-training）**。它是多模态领域的经典之作，后续也作为基础模型，被广泛用在DALI2，Stable Diffusion等重要文生图大模型中。话不多说，进入正文～

CV大模型系列文章导航（持续更新中）：

- 🌸 [CV大模型系列之：扩散模型基石DDPM（模型架构篇）](#) 🌸
- 🌸 [CV大模型系列之：扩散模型基石DDPM（人人都能看懂的数学原理篇）](#) 🌸
- 🌸 [CV大模型系列之：扩散模型基石DDPM（源码解读与实操篇）](#) 🌸
- 🌸 [CV大模型系列之：全面解读VIT，它到底给植树人挖了多少坑](#) 🌸
- 🌸 [CV大模型系列之：多模态经典之作CLIP，探索图文结合的奥秘](#) 🌸
- 🌸 [CV大模型系列之：MAE，实现像素级图像重建](#) 🌸
- 🌸 [CV大模型系列之：MoCo v1，利用对比学习在CV任务上做无监督训练](#) 🌸

一、CLIP在做一件事

在使用ViT做传统图像分类的过程中，我们的训练是“有标签的”。如下图所示，每张输入数据都是 <image, label> 的形式，最终我们用MLP Head位置上对应的向量，来做图片的类别预测。



这样的设计有2个显著缺点：

- **缺点1：**如果出现了一张图，其中包含模型从来没见过的类别，那么模型就不能输出正确的结果。（例如，训练时用的是动物图片，预测时给模型一张汽车图片）
- **缺点2：**如果输入数据出现了分布偏移（distribution shift），那么模型可能也无法输出正确的结果。（例如，缺点1中描述的算一种偏移，另外训练时用的是正常的动物图片，预测时给的是毕加索风格的动物图片也算一种偏移）

解决这2个缺点的传统方法是：微调。但是多模态却想做一步到位的事情：不用做任何微调，也能实现 zero-shot 的图片分类。

对于缺点1来说，zero-shot是指，你给我一串标签 <dog>, <cat>....<car>，即使训练数据中从没有出现过汽车图片（zero-shot，一张都没命中），当我喂一张汽车图片时，模型能告诉我属于 <car>（图->文）。或者说，当我让模型从一堆图片里找出 <car> 的时候，它也能准确地找到（文->图）。

对于缺点2来说，zero-shot是指，我的训练数据中从没毕加索风格的动物图片，我只给模型喂正常的动物图片。但是在测试阶段，模型在毕加索风格的动物图片上的准确率依然不错。在CLIP的实验过程

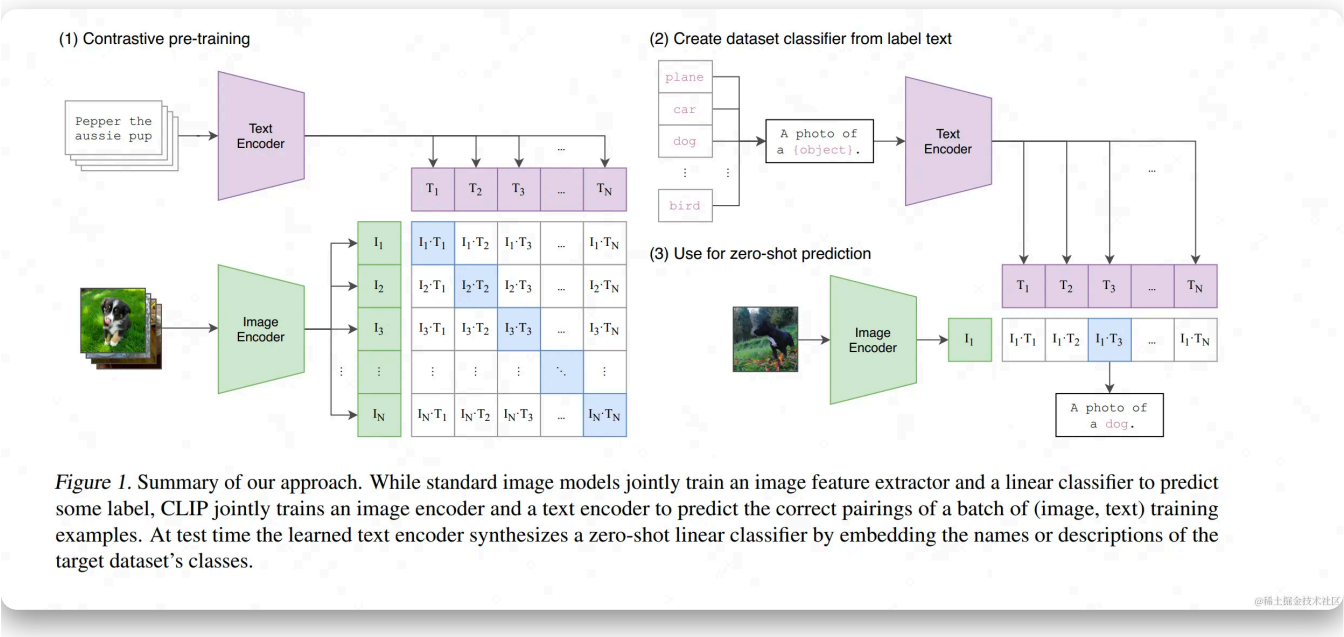
在我个人看来，CLIP解决缺点2的意义，要高于缺点1。因为对缺点1来说，只要训练数据集够大，那么模型是能做排除法的。而对缺点2，却意味着模型不仅要能提炼出不同模态数据中的关键特征，还要真正掌握这些特征间的相关性。同时，在现实世界中，文字分类基本是固定的，但图像内容却可以千变万化。

当然了，CLIP的作用也不止于单纯的图像分类，例如传统的OCR识别、视频中的动作识别等任务，都可以用相似的原理来实现，只需要在训练/预测时修改文字输入的prompt即可。我们会在下文中来看这一点。

好，说明了CLIP要实现的目的后，我们接下来看看，它是通过什么办法，来达到这个目的的。

二、CLIP整体架构

2.1 CLIP的训练



图中（1）部分刻画了CLIP的预训练过程，我们来详细解读下。

2.1.1 训练数据

CLIP的训练数据是 <图像, 文本> pair。如图所示，一个batch的数据里，有若干张图像，每张图像都配有相应的文字描述信息（prompt），比如：

- 一张小狗图片，prompt为 <dog> ，或者为 <A photo of a dog>

值得一提的是，CLIP的作者发现，prompt的设计也会影响模型最终的效果，比如：

- 把prompt从单词 <dog> 换成句子 <A photo of a dog> 后，模型在ImageNet分类任务上的准确率直接提高了1.3%
- 在**OCR数据集**上，作者发现如果把要识别的文字、数字用引号扩起来，能达到更好的效果
- 在**卫星图分类数据集**上，作者发现把prompt替换成 <A satellite photo of a house>，效果会更好
- 在设计到多语义的场景，比如crane既可以表示仙鹤，又可以表示起重机。这时如果把prompt写成 <A photo of a crane, a type of pet>，就能解决歧义问题。

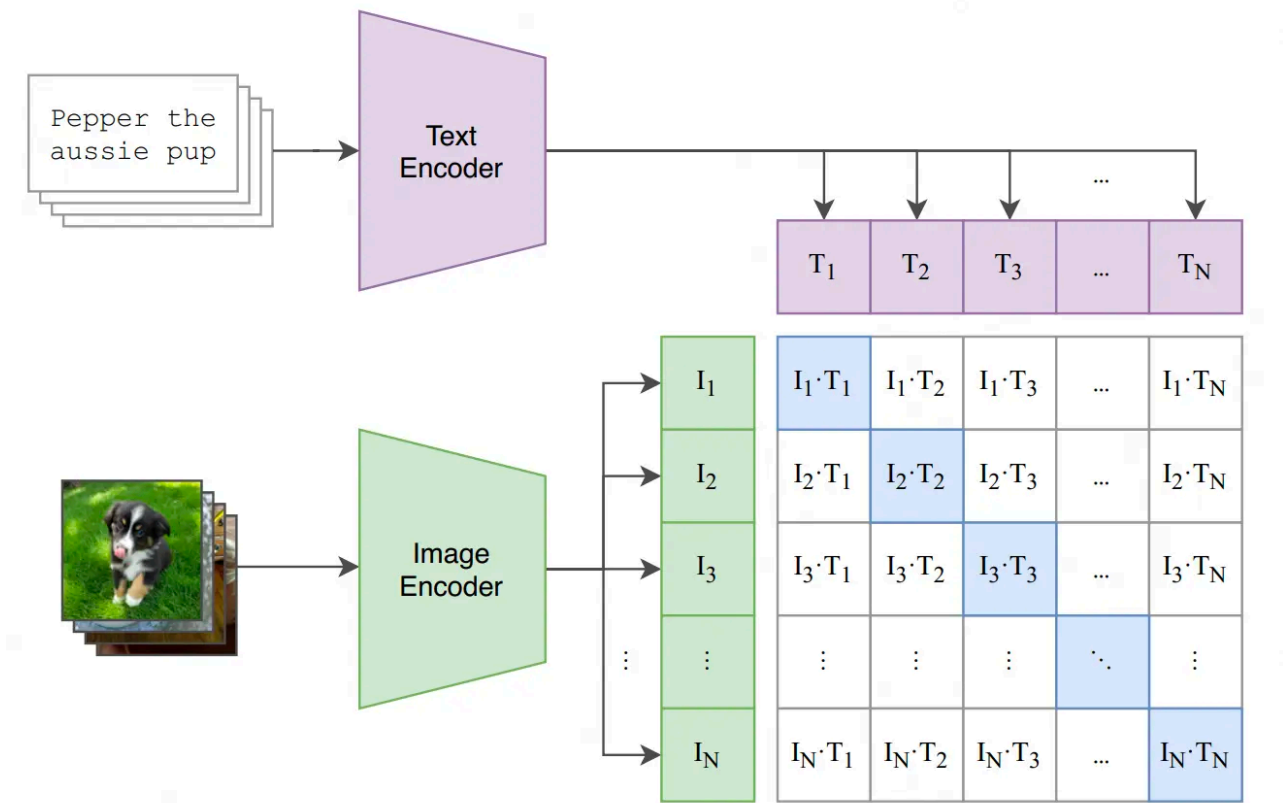
在论文的3.1.4部分，还有关于prompt工程的详细讨论，感兴趣的朋友，可以详读。

在训练中，CLIP没有用前人已经做好的“图像-文本”数据集，因为一来这些数据集质量不高，二来数量太少。CLIP团队自己动手，制作了一个含4亿“图像-文本”对的数据集。制作的方法是，首先从Wikipedia上取出出现次数在100以上的词制作成一个query list，然后保证其中每个query都有约2w个“图像-文本”对。

好，介绍完了数据集，我们可以来看CLIP的训练方法了。

2.1.2 CLIP预训练方法：对比学习

(1) Contrastive pre-training



@稀土掘金技术社区

CLIP模型由两个主体部分组成：Text Encoder和Image Encoder。这两部分可以分别理解成文本和图像的特征提取器。

对于Text Encoder，CLIP借鉴的是GPT2（Radford et al.2019）的架构。对于每条prompt，在进入Text Encoder前，都会添加表示开始和结束的符号 [SOS] 与 [EOS]。最终将最后一层 [EOS] 位置的向量作为该prompt的特征表示向量，也就是图中所绘的 T_i 。

对于Image Encoder，CLIP则尝试过5种不同的ResNet架构和3种ViT架构，最终选用的是“ViT-L/14@336px”这个模型，也就是架构为Large，patch_size = 14的ViT，同时在整个CLIP预训练结束后，用更高分辨率（336*336）的图片做了一个epoch的fine-tune，目的是让CLIP能涌现出更好的效果。与Text Encoder类似，每张图片对应一个最终特征表示向量 I_i 。在读论文的过程中，我没有发现 I_i 是来自于哪一出入层位置（也可能是我读漏了），但我猜测应该和Text Encoder差不多，可能来自分类头 [CLS]。

需要注意的是，CLIP是从头开始训练它的Text Encoder和Image Encoder的，没有借助其余预训练结果。

对比学习

假设一个batch中共有N对 <图像, 文字> 对，那么它们过完各自的Encoder后，就会分别产生：

- N条文字向量 $[T_1, T_2, \dots, T_N]$
- N条图片向量 $[I_1, I_2, \dots, I_N]$

这两组向量，将会分别过一次多模态Embedding（multimodal embedding），也就是在图中代表文字的紫色向量下，还有一层参数 W_t （图中没有画出来），文字向量需要先和 W_t 做矩阵相乘后，才能得到最终的文字向量。对图片向量，同理也有个对应的 W_i 。 W_t, W_i 的作用可以理解成把文字、图片特征投影到多模态的特征空间中去。

经过多模态Embedding的处理，我们得到了最终的 $[T_1, T_2, \dots, T_N]$ 和 $[I_1, I_2, \dots, I_N]$ 。接下来，我们就能通过“对比学习”，找到图像和文字的相似关系。做法也很简单，对于图中列出的N*N个格子，我们只需计算每个格子上对应的向量点积（余弦相似度）即可。由于对角线上的图片-文字对是真值，我们自然希望对角线上的相似度可以最大，据此我们可设置交叉熵函数，来求得每个batch下的Loss。

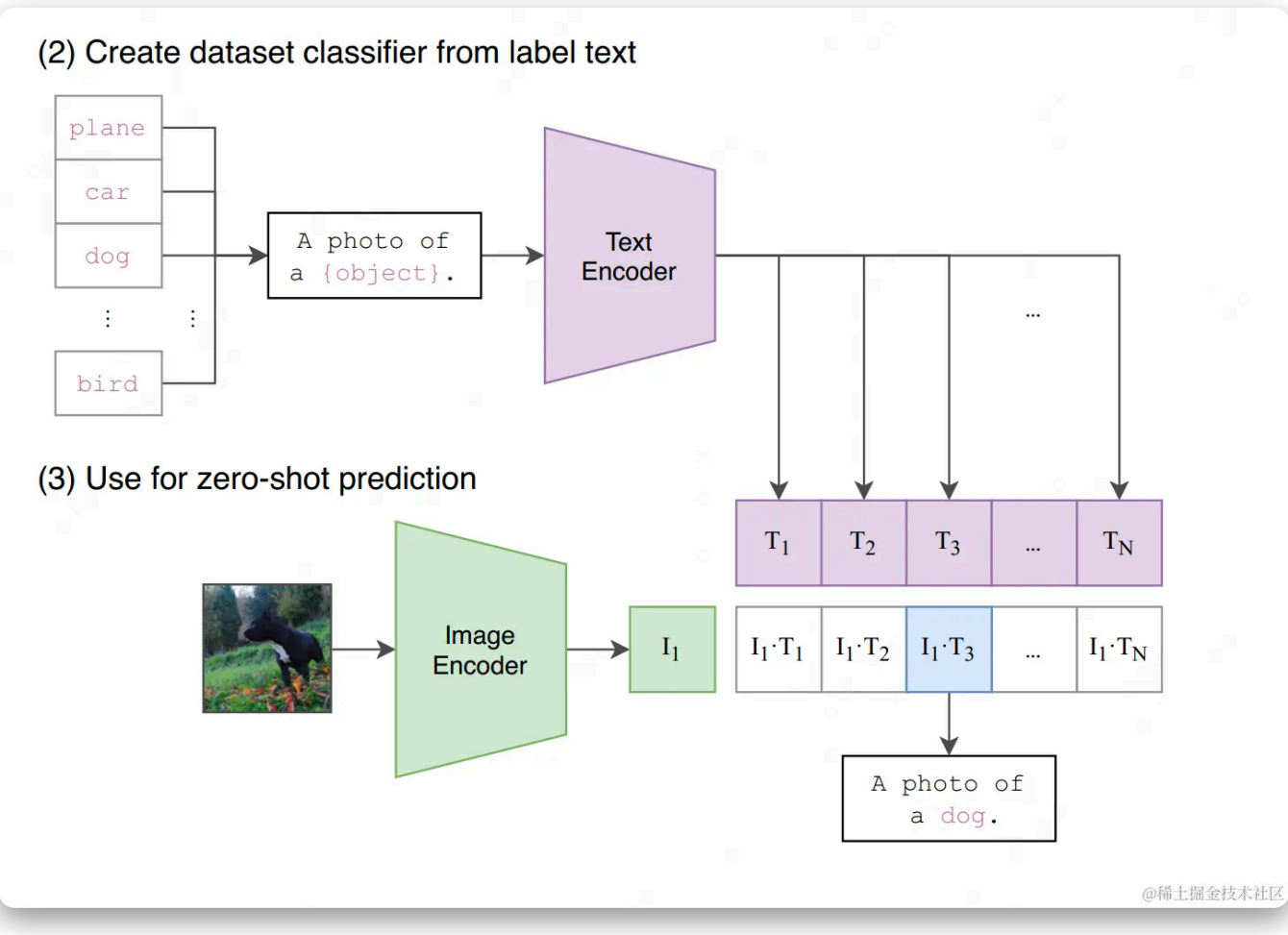
```
1 # image_encoder - ResNet or Vision Transformer
2 # text_encoder - CBOW or Text Transformer
3 # I[n, h, w, c] - minibatch of aligned images
4 # T[n, l] - minibatch of aligned texts
5 # W_i[d_i, d_e] - learned proj of image to embed
6 # W_t[d_t, d_e] - learned proj of text to embed
7 # t - learned temperature parameter
8 # extract feature representations of each modality
9
10 # -----
11 # 1、图像/文字数据过image/text encoder，提取单模态特征
12 # 每张图片对应一个基本特征I_i
13 # 每张文字对应一个基本特征T_i
14 # -----
15 I_f = image_encoder(I) #[n, d_i]
16 T_f = text_encoder(T) #[n, d_t]
17
18 # -----
19 # 2. 图像/文字的基本特征过多模态Embedding，提取多模态特征
20 # 同时对这两个多模态特征做Layer Norm
21 # -----
22 I_e = l2_normalize(np.dot(I_f, W_i), axis=1) # [n, d_i] * [d_i, d_e] = [n, d_e]
23 T_e = l2_normalize(np.dot(T_f, W_t), axis=1) # [n, d_t] * [d_t, d_e] = [n, d_e]
24
25 # -----
26 # 3、计算图片-文字向量的余弦相似度
27 # -----
28 logits = np.dot(I_e, T_e.T) * np.exp(t) # [n, n]
29
30 # -----
31 # 4、计算Loss
32 # -----
33 labels = np.arange(n)
34 loss_i = cross_entropy_loss(logits, labels, axis=0)
35 loss_t = cross_entropy_loss(logits, labels, axis=1)
36 loss = (loss_i + loss_t)/2
```

很多朋友可能对最后一步计算Loss有迷惑，搞不懂为什么要算两个Loss再取平均，这里解释一下：

- CLIP分为**按行计算Loss**和**按列计算Loss**
- **按行计算Loss**，在每一行范围内做softmax，然后计算cross_entropy（蓝色格子部分是真值）。这样计算Loss的意义是：对于每一张图片，我们都希望找到和它最相似的文字。

- **按列计算Loss**，在每一列的范围内做softmax，然后计算cross_entropy（蓝色格子部分是真值）。这样计算Loss的意义是：对于每一段文字，我们都希望找到和它最相似的图片。
- **最后将这两个Loss相加取平均**，代表我们在模型优化过程中考虑了“图片->文字”和“文字->图片”的双向关系。

2.1.3 CLIP Zero-shot预测



当我们做完模型的预训练后，就能用模型来做之前说的zero-shot预测了，方法也非常简单：

- 首先，我们创建一个标签全集，如图中（2）所示，并得到每一个标签的特征向量
- 然后，我们取一张图片，如图中（3）所示，过Image Encoder后得到该图片的特征向量
- 最后，计算图片向量和文字向量间的相似度，取相似度最高的那条label即可。

代码实现如下：

Python 复制代码

```
1 import os
2 import clip
3 import torch
4 from torchvision.datasets import CIFAR100
5
6 # -----
7 # 1、读取模型
8 # -----
9 device = "cuda" if torch.cuda.is_available() else "cpu"
10 model, preprocess = clip.load('ViT-B/32', device)
11
12 # -----
13 # 2、下载数据集
14 # -----
15 cifar100 = CIFAR100(root=os.path.expanduser("~/cache"), download=True, train=False)
16
17 # -----
18 # 3、 (1) 从数据集中随机抽取一张图片，作为图片输入
19 #      (2) 取出该数据集下所有的标签，作为文字数据
20 # -----
21 image, class_id = cifar100[3637]
22 image_input = preprocess(image).unsqueeze(0).to(device)
23 text_inputs = torch.cat([clip.tokenize(f"a photo of a {c}") for c in cifar100.classes]).
24
25 # -----
26 # 4、计算图像、文字的特征向量
27 # -----
28 with torch.no_grad():
29     image_features = model.encode_image(image_input)
30     text_features = model.encode_text(text_inputs)
31
32 # -----
33 # 5、分别对图像、文字特征向量做归一化处理，
34 #     然后计算余弦相似度
35 #     取最相似的top5结果
36 # -----
37 image_features /= image_features.norm(dim=-1, keepdim=True)
38 text_features /= text_features.norm(dim=-1, keepdim=True)
39 similarity = (100.0 * image_features @ text_features.T).softmax(dim=-1)
40 values, indices = similarity[0].topk(5)
41
42 # -----
43 # 6、打印结果
44 # -----
45 print("\nTop predictions:\n")
46 for value, index in zip(values, indices):
```


在读Zero-shot预测的代码中，你可能已经发现，对于标签来说，CLIP需要一个标签全集。也就是说，当你喂给CLIP一张图时，不管这张图片它是否有见过，CLIP都不会生成一个全新的标签，而是去全集标签中找一个最相似的给你（其实，这也是CLIP的缺陷之一，在论文的后面有做讨论）。借助这个代码，我们可以更好理解CLIP zero-shot的含义，也可以更好理解前文所说：只要训练数据集够大，模型总有办法做排除法的含义。

三、CLIP的缺陷

到目前为止，我们已经把CLIP技术部分讲完了，怎么样，是不是比想象中的简单多了？虽然技术简单，但CLIP的论文肝了48页，来分析各种实验效果和其训练代价（CLIP训起来也是很贵）。因此，我这里就不花篇幅去介绍这两块了，感兴趣的朋友可以看看论文。

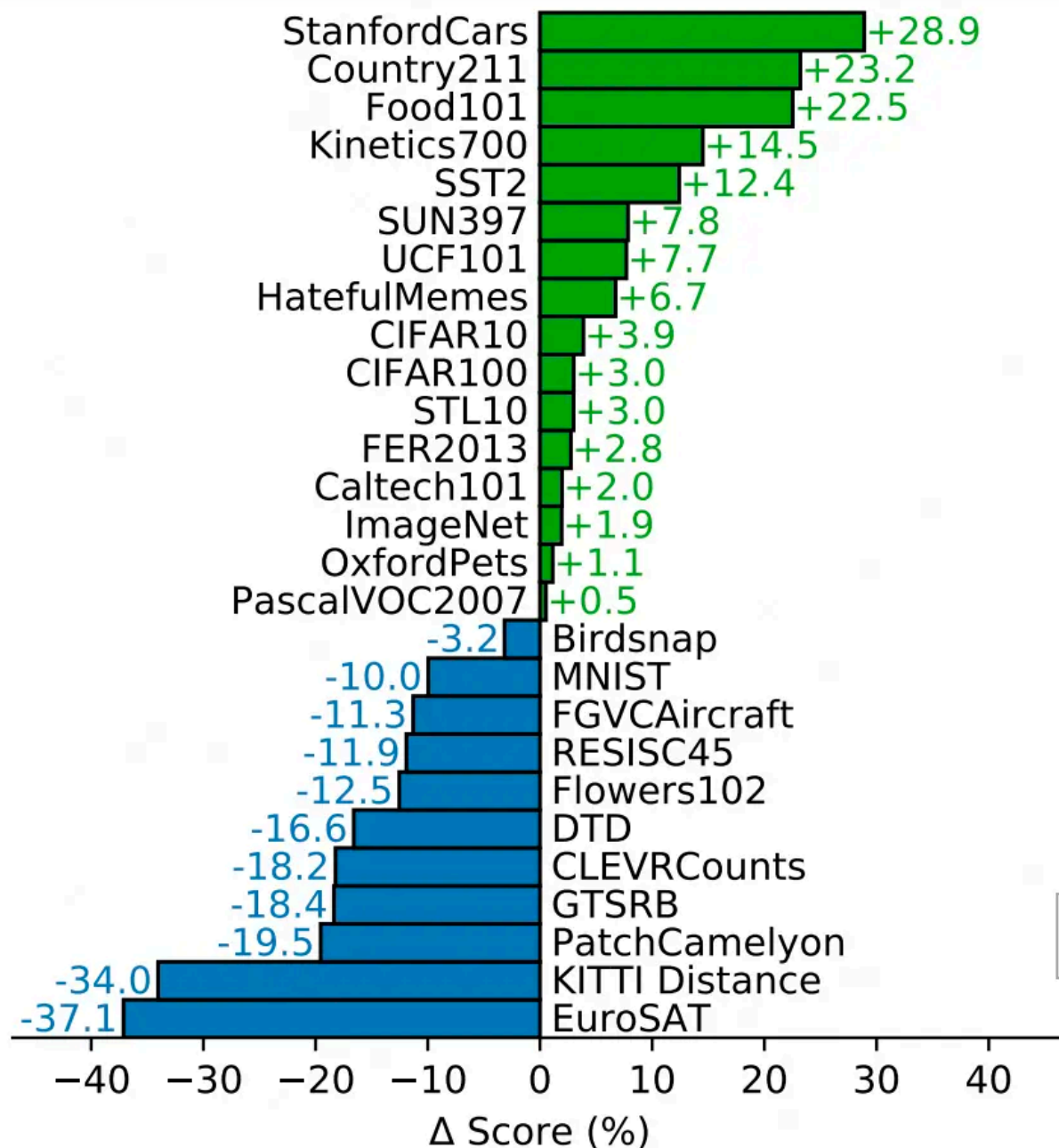
在这里我们想讨论的，是CLIP这个厉害模型，到底存在哪些缺陷。

缺陷一：Zero-shot的能力很强，但不是最强的。

根据实验结果，CLIP从来没有用ImageNet的数据训练过，但它在ImageNet上的预测效果可以达到76.2%，和用ImageNet做训练集的ResNet50基本一致。乍看之下，CLIP的表现很不错了。但其实，ResNet50并不是在ImageNet分类任务上表现最SOTA的模型，例如MAE之类在ImageNet上可以达到80%+。虽然CLIP同样具有涌现能力，即当模型变大时，模型的效果会更好，但是因为CLIP训练昂贵的原因，为了提升预测百分点而需要的代价是巨大的。因此这也是CLIP当前的限制之一。

缺陷二：CLIP无法处理更抽象的任务。

抽象的任务指：输出图片中物体的个数等需要一定逻辑思维推理的任务。在论文的实验中也有给出一些说明，下图中刻画了CLIP和ResNet在不同数据集任务上的表现情况。绿色表示CLIP表现更好的数据集，蓝色表示ResNet表现更好的数据集。注意到蓝色部分的DTD（纹理分类）和CLEVRCountS（给图中物体计数）这两个数据集，都是相对抽象的任务，在这方面CLIP的表现明显不如ResNet。



Zero-Shot CLIP vs. Linear Probe on ResNet50

Figure 5. Zero-shot CLIP is competitive with a fully supervised baseline. Across a 27 dataset eval suite, a zero-shot CLIP classifier outperforms a fully supervised linear classifier fitted on ResNet-50 features on 16 datasets, including ImageNet.

@稀土掘金技术社区

缺陷三：当测试数据集分布严重偏移时，CLIP也束手无策。

虽然CLIP以Zero-shot标榜，但是如果测试数据集分布相对训练数据集分布存在严重偏移情况时，CLIP的表现也不理想。论文中提出了一个很有代表性的例子：MNIST（手写数字数据集）。这样一个简单的数据集，可能用SVM都能做到99%以上的准确率了，但CLIP在上面的表现只有99%，原因就是左

缺陷四：文字标签是个闭集。

前文说过，在对CLIP做zero-shot预测时，我们的文字标签是一个闭集，模型吃一张可能没有见过的图片，然后从这个闭集中找出最匹配的标签，而不是去预测出一个新的文字标签。从这一点上说，CLIP依然不够自动化。

缺陷五：受限于计算资源，无法做图像-文本的生成式网络。

这个在CLIP看来是缺陷的问题，不久之后已经被我们熟知的DALLE2，Stable Diffusion解决了（没错，正是采在CLIP的肩膀上）。因此这是CLIP的限制，但也是后人研究的启发点。

四、参考

1、[arxiv.org/abs/2103.00...](https://arxiv.org/abs/2103.00020)

2、github.com/OpenAI/CLIP

标签： 人工智能 LLM 计算机视觉

评论 8



平等表达，友善交流



0 / 1000 ? 发送

最热 最新



xqcoco LV.3

CV

6月前 点赞 评论



盐酸盐可以倒着念 学生





程序员阿菜：测试

6月前 点赞 回复

...



程序员阿菜 回复 程序员阿菜：测试

6月前 点赞 回复

...

查看全部 3 条回复



提姆队长 LV.2 前端 @韵达科技

1

1

1

6月前 点赞 评论

...

查看全部 8 条评论

目录

收起

一、CLIP在做一件事

二、CLIP整体架构

2.1 CLIP的训练

2.1.1 训练数据

2.1.2 CLIP预训练方法：对比学习

2.1.3 CLIP Zero-shot预测

三、CLIP的缺陷

四、参考

相关推荐

RAG 与微调在大模型应用中如何抉择

1.2k阅读 · 2点赞



基于Transformer的图像分类网络Vit

1.7k阅读 · 3点赞

提示词（prompt）工程指南（六）：对抗提示

978阅读 · 2点赞

Transformer模型-4-Inputs

502阅读 · 2点赞

为你推荐

ChatGPT技术解析之：GPT写代码的能力从何而来

猛猿11月前5.5k256

ChatGPT

CV大模型系列之：全面解读VIT，它到底给植树人挖了多少坑

猛猿8月前4.1k187

人工智能计算机...AIGC

CV大模型系列之：扩散模型基石DDPM（人人都能看懂的数学原理篇）

猛猿8月前4.0k228

人工智能计算机...AIGC

CV大模型系列之：扩散模型基石DDPM（模型架构篇）

猛猿8月前3.8k175

人工智能计算机...AIGC

CV大模型系列之：GAN，博弈论下的一个实例

猛猿5月前2.1k5716

人工智能计算机...LLM

CV大模型系列之：扩散模型基石DDPM（源码解读与实操篇）

猛猿7月前2.7k84

人工智能LLM计算机...

ChatGPT技术解析之：训练框架InstructGPT

猛猿11月前2.1k172

ChatGPTGPT算法

ChatGPT技术解析之：GPT1、GPT2与GPT3

猛猿11月前2.1k9评论

ChatGPT

CV大模型系列之：DALIE2，OpenAI文生图代表作解读

猛猿6月前2.1k5评论

人工智能计算机...LLM

CV大模型系列之：MoCo v1，利用对比学习在CV任务上做无监督训练

猛猿6月前1.8k102

人工智能计算机...LLM

CV大模型系列之：MAE，实现像素级图像重建

猛猿6月前1.4k116

人工智能计算机...LLM

CV大模型系列之：打败ViT？Swin Transformer是怎么做到的

猛猿5月前1.6k4评论

人工智能计算机...LLM

图解Transformer系列一：Positional Encoding（位置编码）

猛猿9月前1.3k6评论

算法人工智能

图解Transformer系列二：Self-Attention（自注意力机制）

猛猿9月前1.2k47

算法人工智能

图解Transformer系列三：Batch Normalization & Layer Normalization（批量&层标准化）

猛猿9月前85611

算法人工智能